

Motivation

In type-theoretic frameworks for complexity analysis, there are two important design considerations: *expressiveness*, the class of programs which the framework can reason about, and *usability*, the degree to which the framework automates the analysis, or relies on user-supplied proofs. The choice of framework can have a significant impact on both of these factors. Our goal is to create a framework which provides a middle ground between these two properties.

Examples

Consider the map function on lists. Intuitively, one expects that the time to map a function f onto a list xs should be the sum of the times to evaluate the application $f\ x$ for each $x \in xs$. We would like for our framework to naturally handle higher-order arguments, and allow for a proof of the following proposition for all $f : A \rightarrow B$ and $xs : \text{List } A$. Terms surrounded by $\langle \cdot \rangle$ can be read as unevaluated/held computations, and terms surrounded by $| \cdot |$ can be read as fully evaluated/forced computations.

$\text{time}(\langle \text{map } f \mid |xs| \rangle) = \text{len } xs + \text{sum } (\text{map } (\text{time}(\langle f \mid | _ | \rangle)) xs)$
Ideally, the following judgemental equalities hold for the empty list $[]$ and the extended list $x : xs$, allowing the above proposition to be shown by induction.

$$\text{time}(\langle \text{map } f \mid |[]| \rangle) \equiv 0$$

$$\text{time}(\langle \text{map } f \mid |x : xs| \rangle) \equiv 1 + \text{time}(\langle f \mid |x| \rangle) + \text{time}(\langle \text{map } f \mid |xs| \rangle)$$

With sufficient infrastructure, our framework should allow one to perform worst-case asymptotic analysis on more complex functions, such as sorting algorithms, and prove statements such as the following worst-case bound for a polymorphic sorting function applied with a constant-time comparison function f .

$$\text{time}(\langle \text{sort } f \mid |xs| \rangle) \leq c * (\text{len } xs) * \log (\text{len } xs)$$

Main Results

- Introduce an extension of the calculus of constructions (CC) [2] which allows for automatic runtime calculation for a wide class of computations
- Allow for natural handling of runtimes for higher-order functions
- Show preservation and strong normalisation
- Propose a generic construction to add runtime reasoning to a variety of languages

Syntax

Our base type theory is the calculus of constructions with an impredicative type **Prop**, and a predicative hierarchy **Type_i**, along with the natural number type **Nat** : **Type**. To this calculus we add a type family of *computations*, **Comp**(A), which represent possibly non-normalised terms of type A in the base CC. This type has the constructors **pure** and **liftA** (denoted as $| \cdot |$ and $\langle \cdot \rangle$ respectively in the above examples) and an evaluation function **eval** to extract the value represented by the computation. The runtime of a computation can be evaluated with the function **time**.

Var $\ni x, y, f, A, \dots$ (variables)
Kind $\ni \kappa ::= \text{Prop} \mid \text{Type}_i, i \in \mathbb{N}$ (kinds)
Kind' $\ni \kappa' ::= \text{Prop}' \mid \text{Type}'_i, i \in \mathbb{N}$ (lifted kinds)
Sort $\ni s ::= \kappa \mid \kappa'$ (sorts)
Exp $\ni e ::= s \mid x \mid \forall x : e.e \mid \lambda x : e.e \mid$ (expressions)
 $e \mid \text{Nat} \mid 0 \mid \text{S}(e) \mid e + e \mid \text{Nat-ind}(e, e, e) \mid$
 $\text{Comp}(e) \mid \text{pure}(e) \mid \text{liftA}(e, e) \mid$
 $\text{Nat-comp}(e, e, e) \mid \text{eval}(e) \mid \text{time}(e)$

Typing

We summarize our universe and product typing as the following pure type system [1] $(\mathcal{S}, \mathcal{A}, \mathcal{R})$, where the sorts $*_0, *_1, i_0$ and i_1 represent **Prop**, **Prop'**, **Type_i**, and **Type'_i**, respectively.

$$\mathcal{S} = \{*_0, *_1, i_0, i_1\}$$

$$\mathcal{A} = \{(*_x, 1_x), (i_x, (i + 1)_x)\}$$

$$\mathcal{R} = \{(*_0, *_0, *_0), (i_0, *_0, *_0), (*_0, j_0, j_0), (i_0, j_0, (i \sqcup j)_0)\}$$

The type family of computations has kinds as its domain, and lifted kinds as its codomain. **liftA** has a fully dependent type, which can be restricted to a standard applicative functorial type.

$$\frac{\Gamma \vdash e : A : \kappa}{\Gamma \vdash \text{pure}(e) : \text{Comp}(A) : \kappa'}$$

$$\frac{\Gamma \vdash A \rightarrow B : \kappa \quad \Gamma \vdash e_1 : \text{Comp}(A \rightarrow B) \quad \Gamma \vdash e_2 : \text{Comp}(A)}{\Gamma \vdash \text{liftA}(e_1, e_2) : \text{Comp}(B)}$$

Semantics

We use a small-step model to define the operational semantics of our language. Our reduction strategy is call by value, where reductions are performed inside of abstractions and other constructors such as **pure** and **Comp**. We add a syntactic construct τ in order to compute term runtimes, where the runtime of open terms is interpreted through the **time** function.

$$\text{time}(\text{pure}(v)) \Rightarrow 0$$

$$\text{time}(\text{liftA}(v_1, v_2)) \Rightarrow \tau[\text{eval}(v_1) \text{ eval}(v_2)]$$

$$\frac{e \Rightarrow e'}{\tau[e] \Rightarrow 1 + \tau[e']}$$

$$\tau[x \ v] \Rightarrow \text{time}(\text{liftA}(\text{pure}(x), \text{pure}(v)))$$

Strong Normalisation

Proof idea for strong normalisation: define a semantics-preserving map into the calculus of inductive constructions (CIC) [3].

- We assume that the CIC is strongly normalizing, and that there is an interpreter for the CC in the CIC
- The forms of quantification defined by $\mathcal{R}$ ensure that every term in our language is a term of the CC, or the runtime of a possibly open term of the CC
- Map CC terms to their representation in the interpreter, and runtimes to the runtime evaluation function applied to the representation

Conclusions

We have defined a calculus which allows for natural evaluation and reasoning for runtimes for a large class of functions, those definable by the CC, and natural handling of higher-order runtimes. By appealing to the strong normalisation of the CIC, and the existence of an interpreter for the CC in the CIC, we can show strong normalisation for this calculus. A form of preservation/subject reduction can also be shown, leading to the internal consistency of the calculus. It is an interesting question to consider which languages can be extended with runtime reasoning in an analogous manner. The key ingredients to our construction are dependent types, and an interpreter in a strongly normalising language, so it seems that this construction can be applied to a wide variety of languages.

References

- [1] Henk (Hendrik) Barendregt. "An Introduction to Generalized Type Systems". In: *Journal of Functional Programming* 1 (Apr. 1991), pp. 125–154. DOI: 10.1017/S0956796800020025.
- [2] Thierry Coquand and Gérard Huet. "The calculus of constructions". In: *Information and Computation* 76.2 (1988), pp. 95–120. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3). URL: <http://www.sciencedirect.com/science/article/pii/0890540188900053>.
- [3] Christine Paulin-Mohring. "Introduction to the Calculus of Inductive Constructions". In: (Nov. 2014).